ONE THREAD · DENSO FACTORY HACKS 2026 · ĐỀ BÀI D3

Kiến trúc hệ thống

Dữ liệu nhà máy đi từ hệ thống của từng phòng ban tới AI agent như thế nào, để khi có lỗi có thể truy ngược về đúng máy, đúng lô và đúng tài liệu liên quan.

Phiên bản 1 · 08/11/2026 · hợp nhất hai bản phác thảo pipeline của đội

Tóm tắt

One Thread đọc dữ liệu từ sản xuất, sản phẩm và chất lượng, các hệ thống kinh doanh và tài liệu tự do, mà không thay đổi bất kỳ hệ thống nào. Mỗi bản ghi được đưa về một khuôn event chung, được gán một ID duy nhất cho mỗi đối tượng nghiệp vụ (máy, lô, linh kiện, nhà cung cấp, đơn hàng, khách hàng, nhân viên...), rồi đặt lên một đồng hồ chung của nhà máy. Bước liên kết sau đó nối các bản ghi lại với nhau theo thang độ tin cậy năm bậc, từ khớp khoá chính xác cho tới các gợi ý cần kỹ sư xác nhận. Kết quả là một *digital thread*: kho chứa đối tượng, event, tài liệu và liên kết, trả lời được câu hỏi "linh kiện này đã trải qua những gì, ở đâu, lúc nào, và tài liệu nào mô tả nó?"

Phía trên thread có ba lớp dữ liệu tối ưu cho việc đọc: trạng thái live cho các màn hình nhà máy, lakehouse cho lịch sử và KPI, và knowledge graph cho truy vết nhiều bước. Một lớp truy vấn duy nhất phục vụ cả ba, có phân quyền và audit log, đồng thời mở dữ liệu cho AI agent dưới dạng các MCP tool chỉ đọc. Agent trả lời câu hỏi và soạn phân tích nguyên nhân, trong đó mọi nhận định đều dẫn về bản ghi gốc.

Nguyên tắc thiết kế

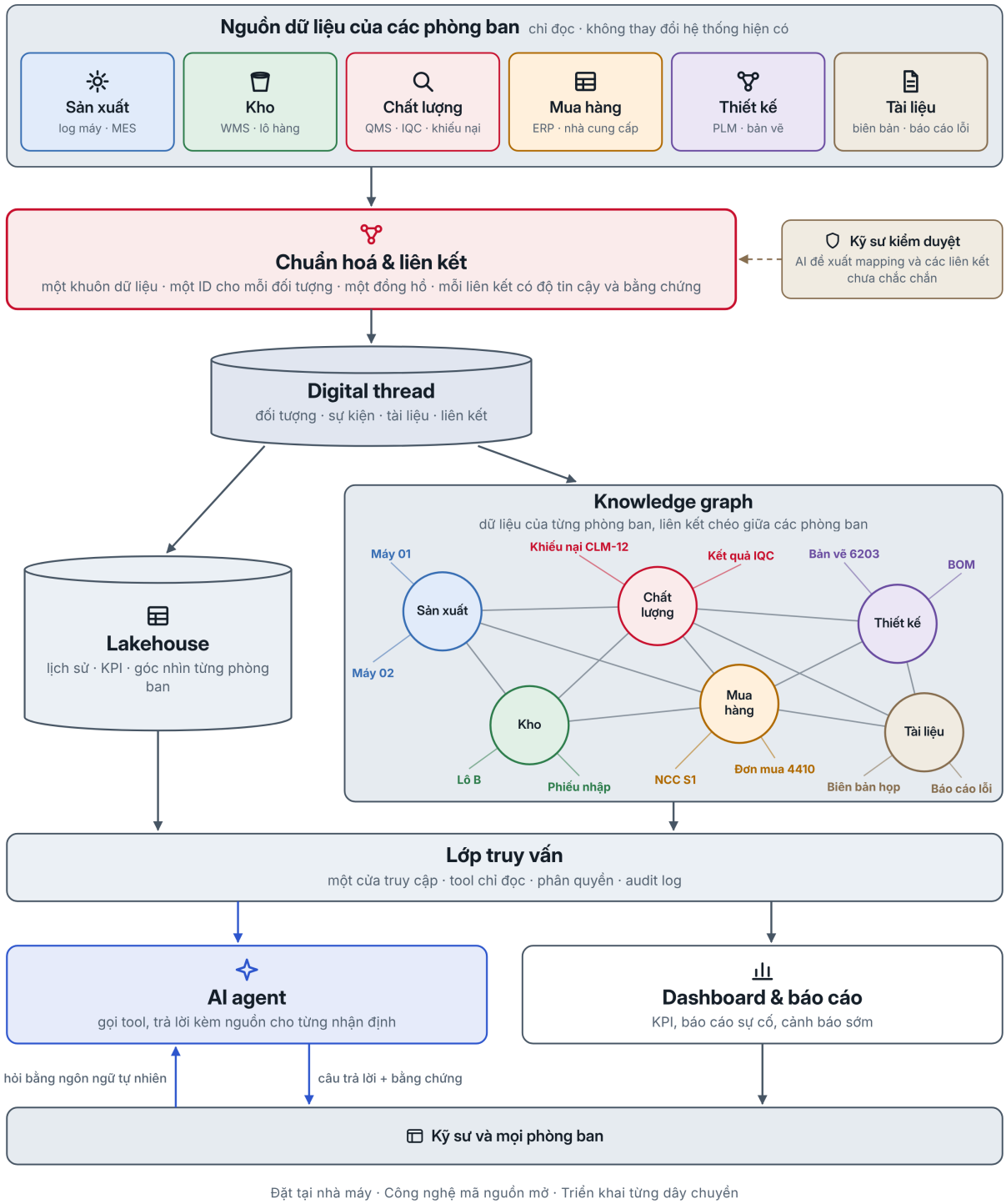
1. **Chỉ đọc ở nguồn.** Connector thích nghi với từng hệ thống. Không phòng ban nào phải sửa phần mềm hay gửi dữ liệu theo định dạng mới.
2. **Giữ bản sao gốc.** Mọi thứ được lưu nguyên như khi nhận, nên có thể dựng lại thread khi mapping hoặc luật liên kết được cải thiện.
3. **Chuẩn hoá trước, liên kết sau.** Khuôn dữ liệu, định danh và thời gian được xử lý ở ba bước riêng, mỗi bước dựa trên một registry có quản lý version.
4. **Liên kết nào cũng có bằng chứng.** Mỗi liên kết ghi rõ luật nào tạo ra nó, độ tin cậy bao nhiêu và đã có người xác nhận hay chưa.
5. **AI chỉ đọc và luôn dẫn nguồn.** Agent làm việc qua các tool chỉ đọc và không được khẳng định điều gì mà nó không chỉ ra được nguồn.
6. **AI đề xuất, con người duyệt.** Mapping, bí danh và các liên kết yếu do AI đề xuất và kỹ sư phê duyệt.

Mục lục

1. Sơ đồ kiến trúc
2. Từng lớp của hệ thống
3. Control plane
4. Mô hình dữ liệu lõi
5. AI agent và các tool
6. Kịch bản: truy vết claim sau ba tháng
7. Hợp nhất hai bản phác thảo của đội
8. Phần demo và kiến trúc mục tiêu
9. Tổng hợp công nghệ
10. Các quyết định còn mở

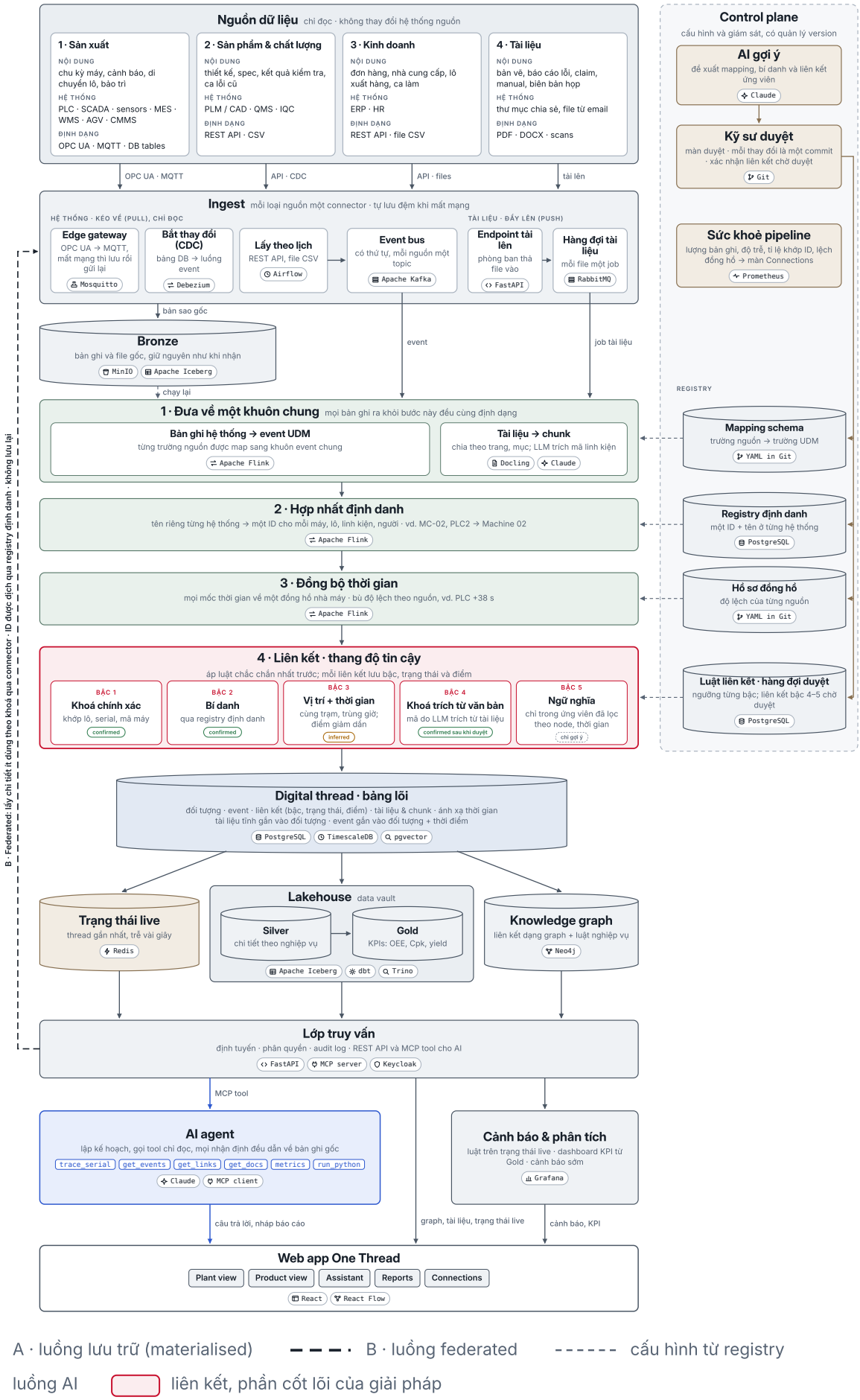
1 Sơ đồ kiến trúc

Hình 1 là bức tranh tổng quan: dữ liệu đi từ hệ thống của các phòng ban, được chuẩn hoá và liên kết thành một digital thread, rồi được AI agent và dashboard đọc qua một lớp truy vấn duy nhất. Hình 2 ở trang sau trình bày chi tiết từng lớp và cả hai luồng dữ liệu.



Hình 1. Tổng quan. Trong knowledge graph, dữ liệu của từng phòng ban treo dưới nút của phòng ban đó; đường nối giữa hai phòng ban nghĩa là dữ liệu của chúng có liên kết.

Hình 2. Pipeline chi tiết



A · luồng lưu trữ (materialised) — — — B · luồng federated ------ cấu hình từ registry

luồng AI liên kết, phần cốt lõi của giải pháp

2 Từng lớp của hệ thống

Ví dụ xuyên suốt lấy từ bộ mockup: Line 1 của bơm nước MH-20, nơi Machine 02 chạy chậm lại sau khi Hopper 1 được nạp Bearing 6203 lô B lúc 13:52.

2.1 Nguồn dữ liệu

Dữ liệu nằm ở bốn nhóm hệ thống. Hệ thống chỉ đọc, không bao giờ ghi vào.

Sản xuất	PLC, SCADA, cảm biến, MES, WMS, đội AGV, CMMS. Chu kỳ máy, cảnh báo, di chuyển lô, bảo trì. OPC UA, MQTT, bảng database.
Sản phẩm & chất lượng	PLM / CAD, QMS, kiểm tra đầu vào (IQC). Thiết kế, spec, kết quả kiểm tra, ca lỗi cũ. REST API, CSV.
Kinh doanh	ERP, HR. Đơn hàng, nhà cung cấp, lô xuất hàng, lịch ca. REST API, file CSV.
Tài liệu	Bản vẽ, báo cáo lỗi, claim khách hàng, manual máy, biên bản họp. PDF, DOCX, bản scan.

2.2 Ingest

Hai cửa vào: một cho hệ thống, một cho tài liệu.

Hệ thống được kéo về (pull), mỗi loại nguồn dùng một connector. **Edge gateway** dưới xưởng đọc PLC qua OPC UA và gửi đi bằng MQTT; khi mất mạng, nó lưu tạm và gửi lại sau. **Bắt thay đổi (CDC)** đọc các dòng mới trong bảng MES và WMS qua log của database, nên hệ thống nguồn không phải gánh thêm tải. **Lấy theo lịch** gọi REST API và lấy file CSV, ví dụ file PLM xuất hằng đêm. Toàn bộ dữ liệu hệ thống đổ về một **event bus**, mỗi nguồn một topic có thứ tự.

Tài liệu được đẩy lên (push). Các phòng ban thả file vào **endpoint tải lên**; mỗi file thành một job trong **hàng đợi tài liệu**. Đây là chỗ hợp lý nhất cho endpoint dạng push, vì tài liệu không có hệ thống nào để kéo về.

CÔNG NGHỆ Mosquitto (MQTT), OPC UA, Debezium, Airflow, Apache Kafka, FastAPI, RabbitMQ

2.3 Bronze: bản sao gốc

Mọi event và mọi file được lưu nguyên như khi nhận, kèm thời điểm nhận và nguồn.

Bronze không bao giờ bị sửa. Mục đích của nó là **chạy lại (replay)**: khi kỹ sư duyệt một bí danh mới (ví dụ mã nhà cung cấp "S1-6203-B" chính là Bearing 6203 rev B), phần lịch sử liên quan được xử lý lại và các bản ghi cũ có thêm những liên kết trước đây chưa nối được.

CÔNG NGHỆ MinIO (object storage kiểu S3 đặt tại chỗ), bảng Apache Iceberg dạng Parquet

2.4 Bước 1 · Đưa về một khuôn chung

Mọi bản ghi ra khỏi bước này đều cùng một định dạng.

Bản ghi hệ thống được map từng trường sang event theo mô hình dữ liệu thống nhất (UDM): việc gì xảy ra, với đối tượng nào, lúc nào, từ bản ghi gốc nào. Mapping của từng nguồn nằm trong registry mapping schema, nên thêm một nguồn mới chỉ cần thêm file mapping, không phải viết code mới.

Tài liệu được parse và chia thành chunk theo trang và mục. LLM đọc từng chunk và trích ra các mã được nhắc tới (mã linh kiện, lô, tên máy). Ví dụ, một file thiết kế MH-20 dài 40 trang sẽ thành một chunk cho bản vẽ vỏ bơm, một cho ổ bi, một cho phớt.

CÔNG NGHỆ các job Apache Flink chạy trên luồng Kafka; Docling để parse tài liệu; Claude để trích mã

2.5 Bước 2 · Hợp nhất định danh

Tên riêng ở từng hệ thống được quy về một ID cho mỗi đối tượng nghiệp vụ, dù thuộc phòng ban nào.

MES ghi "MC-02", PLC gửi "PLC2", bên bảo trì gọi là "Press 2". Registry định danh giữ một đối tượng chuẩn cho mỗi vật thể thật cùng tên của nó ở từng hệ thống, nên cả ba đều quy về `Machine 02`. Tên nào không khớp được sẽ bị đưa vào khu cách ly và hiện trên màn Connections để duyệt.

CÔNG NGHỆ Flink tra cứu registry định danh trên PostgreSQL

2.6 Bước 3 · Đồng bộ thời gian

Mọi mốc thời gian được đưa về một đồng hồ chung của nhà máy.

Mỗi nguồn có một hồ sơ đồng hồ: độ lệch và mức sai số. PLC của Machine 02 chạy nhanh 38 giây, nên 13:52:38 trên PLC thành 13:52:00 giờ nhà máy, đúng thời điểm WMS ghi lần nạp hopper. Không có bước này, luật cửa sổ thời gian ở bước 4 sẽ nối nhầm bản ghi.

CÔNG NGHỆ Flink; hồ sơ đồng hồ lưu dạng YAML trong Git

2.7 Bước 4 · Liên kết theo thang độ tin cậy

Bản ghi được nối vào đối tượng và nối với nhau, áp luật chắc chắn nhất trước. Đây là nơi tạo ra giá trị của giải pháp.

BẬC	LUẬT	TRẠNG THÁI	VÍ DỤ
1	Khoá chính xác: lô, serial, mã máy chuẩn	confirmed	WMS ghi "nạp Hopper 1, lô B" và chu kỳ PLC của part 0418 khớp chính xác lô và serial.
2	Bí danh qua registry định danh	confirmed	"MC-02" và "PLC2" được quy về Machine 02, rồi nối như bậc 1.
3	Cùng trạm và trùng cửa sổ thời gian	inferred	Log rung tại Machine 02 lúc 13:52–14:10, không có serial, được nối với các part gia công ở đó trong khoảng thời gian này. Lệch thời gian càng xa thì điểm càng thấp.
4	Khoá do LLM trích từ văn bản	confirmed sau khi duyệt	Chunk ổ bi trong file thiết kế MH-20 được gắn vào Bearing 6203 sau khi kỹ sư chấp nhận.

BẬC	LUẬT	TRẠNG THÁI	VÍ DỤ
5	Tương đồng ngữ nghĩa, chỉ trong tập ứng viên đã lọc theo loại đối tượng và thời gian	chỉ gợi ý	Một báo cáo lỗi cũ về "tiếng ồn ổ bi" không ghi mã linh kiện được hiển thị như một liên kết có thể có với Bearing 6203. Không bao giờ được dùng như sự thật.

Mỗi liên kết lưu bậc, trạng thái, điểm, version của luật đã tạo ra nó và các bản ghi làm bằng chứng. Liên kết bậc 4 và 5 nằm trong hàng đợi duyệt cho tới khi kỹ sư quyết định. Hình 4 ở mục 4 cho thấy các dòng này trở thành cạnh của knowledge graph như thế nào.

CÔNG NGHỆ Flink cho bậc 1–3, Claude để trích mã cho bậc 4, pgvector để so độ tương đồng cho bậc 5; luật và hàng đợi duyệt trên PostgreSQL

2.8 Digital thread · bảng lõi

Kho dữ liệu gốc. Mọi thứ phía sau đều được dựng từ đây.

Gồm đối tượng, event, tài liệu và chunk, liên kết và ánh xạ thời gian (Hình 3 ở mục 4 minh hoạ sáu bảng kèm dòng ví dụ). **Tài liệu tĩnh** như bản vẽ, spec, kết quả IQC gắn vào một đối tượng. **Dữ liệu động** như log chu kỳ và cảnh báo là event gắn vào một đối tượng tại một thời điểm. Plant view vẽ các event này trên cạnh giữa hai máy, nên hình ảnh "log nằm trên cạnh" của đội vẫn được giữ trên màn hình, trong khi cách lưu trữ vẫn đơn giản.

CÔNG NGHỆ PostgreSQL, kèm TimescaleDB cho chuỗi thời gian của event và pgvector cho embedding của chunk

2.9 Ba lớp dữ liệu tối ưu cho việc đọc

Trạng thái live	Vài giờ gần nhất của thread, trễ chỉ vài giây. Cấp dữ liệu cho Plant view: trạng thái máy, tốc độ trên từng cạnh, vật liệu nào đang cấp vào máy nào. `Redis`
Lakehouse	**Silver** giữ chi tiết đã làm sạch theo từng nghiệp vụ cho toàn bộ lịch sử. **Gold** chứa các view nghiệp vụ và KPI như OEE, Cpk, yield. `Apache Iceberg trên MinIO, dbt, Trino`
Knowledge graph	Liên kết dưới dạng graph, kèm luật nghiệp vụ như "lô đã dùng tại một trạm sẽ tới mọi part gia công ở trạm đó trong khoảng thời gian đó". Trả lời nhanh các câu hỏi nhiều bước: lô → part → thùng → lô xuất hàng. `Neo4j`

2.10 Lớp truy vấn

Một cửa duy nhất cho mọi màn hình và cho AI.

Bộ định tuyến gửi mỗi yêu cầu tới lớp trả lời tốt nhất: trạng thái live cho "hiện tại", lakehouse cho lịch sử và KPI, graph cho truy vết. **Phân quyền** quyết định ai được xem gì, điều này quan trọng vì dữ liệu HR, chi phí và nhà cung cấp giờ chỉ cách một câu truy vấn. **Audit log** ghi lại mọi lần đọc, kể cả của agent. Cùng các chức năng đó được mở ra dưới dạng REST API cho app và **MCP tool** cho agent.

Luồng federated B. Những chi tiết nặng và ít dùng (một bản CAD đầy đủ, bảng chi phí trong ERP) không được sao chép về. Khi có yêu cầu, lớp truy vấn dịch ID chuẩn ngược lại thành tên trong hệ thống nguồn qua registry định danh, rồi lấy dữ liệu qua connector sẵn có. Không lưu lại gì.

CÔNG NGHỆ FastAPI, MCP server, Keycloak cho đăng nhập và phân vai trò

2.11 Ứng dụng

AI agent	Lập kế hoạch cho câu hỏi, gọi tool chỉ đọc, viết câu trả lời và bản nháp báo cáo trong đó mọi nhận định đều dẫn nguồn. Xem mục 5.
Cảnh báo & phân tích	Luật trên trạng thái live phát cảnh báo sớm (cạnh Machine 02 → 03 đang 42 s/part so với mục tiêu 30 s). Dashboard KPI đọc từ Gold. `Grafana`
Web app	Plant view, Product view, Assistant, Reports và Connections. Các màn graph dùng canvas dạng node và cạnh. `React, React Flow`

3 Control plane

Control plane chứa mọi thứ dùng để cấu hình pipeline và cho biết pipeline có đang khoẻ hay không. Mọi loại thay đổi đều đi qua cùng một vòng:

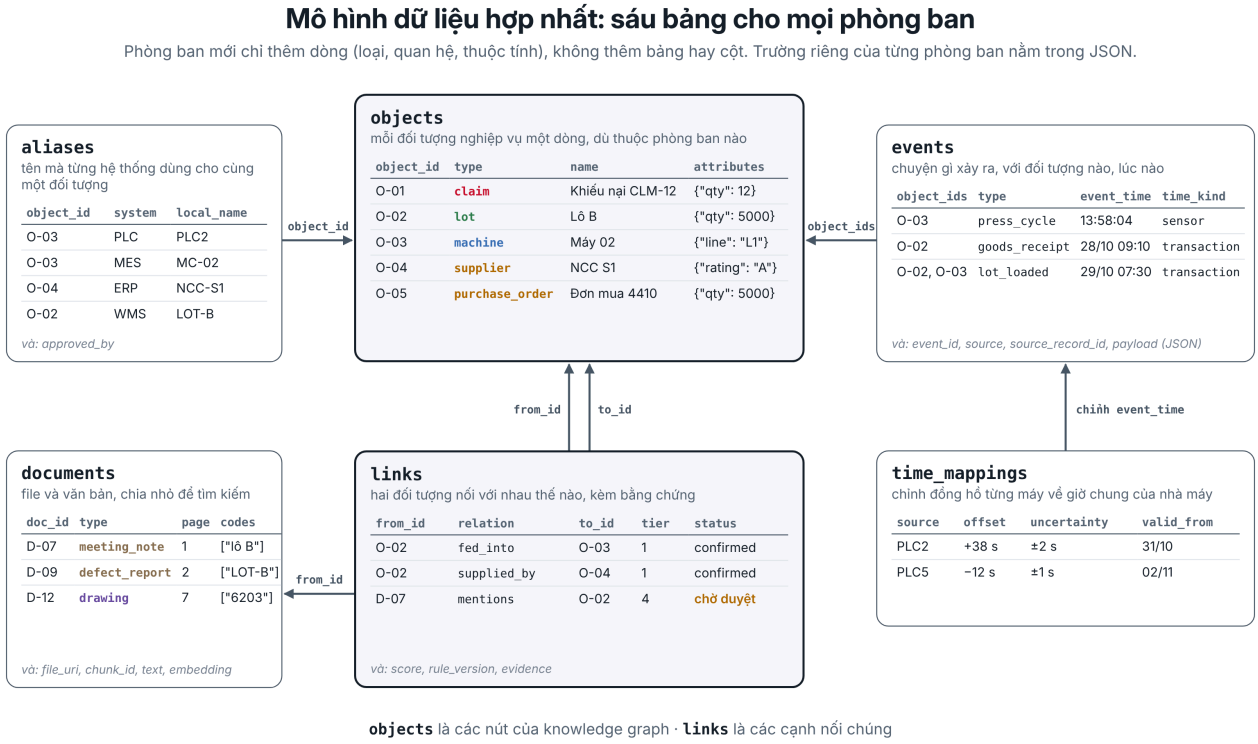
1. **AI đề xuất.** LLM đề xuất mapping cho trường nguồn mới, bí danh cho tên chưa khớp ("PLC3 nhiều khả năng là Machine 03") và các liên kết ứng viên bậc 4 và 5.
2. **Kỹ sư duyệt.** Màn duyệt hiển thị từng đề xuất kèm bằng chứng. Mỗi lần duyệt tạo một commit, nên mọi thay đổi đều có người thực hiện, ngày giờ và có thể hoàn tác.
3. **Registry được cập nhật.** Pipeline đọc version mới; khi cần, Bronze được chạy lại để áp thay đổi cho dữ liệu cũ.

REGISTRY	NỘI DUNG	DÙNG Ở	LƯU TẠI
Mapping schema	Trường nguồn → trường UDM, theo từng nguồn	Bước 1	YAML trong Git
Registry định danh	Một ID chuẩn và tên của nó ở từng hệ thống	Bước 2, luồng federated	PostgreSQL
Danh mục loại và quan hệ	Các loại đối tượng, tên quan hệ được phép, bộ kiểm tra trường theo từng loại	Bước 1–4, AI agent	YAML trong Git
Hồ sơ đồng hồ	Độ lệch và sai số của từng nguồn	Bước 3	YAML trong Git
Luật liên kết, hàng đợi duyệt	Ngưỡng từng bậc, độ rộng cửa sổ thời gian, liên kết bậc 4–5 đang chờ	Bước 4	PostgreSQL

Sức khoẻ pipeline đo lượng bản ghi vào, độ trễ theo nguồn, tỉ lệ khớp ID và độ lệch đồng hồ. Đây chính là các con số trên màn Connections ("Matched to one ID 99.2 %", "ERP trễ 30 phút"), giúp đội dữ liệu biết thread đã đủ trước khi ai đó tin vào câu trả lời. `Prometheus`

4 Mô hình dữ liệu lõi

Mô hình lõi được thiết kế tổng quát: sáu bảng giữ nguyên dù thêm phòng ban hay bài toán nào. Một phòng ban mới chỉ thêm **dữ liệu** (loại đối tượng, tên quan hệ, thuộc tính), không bao giờ thêm bảng hay cột.

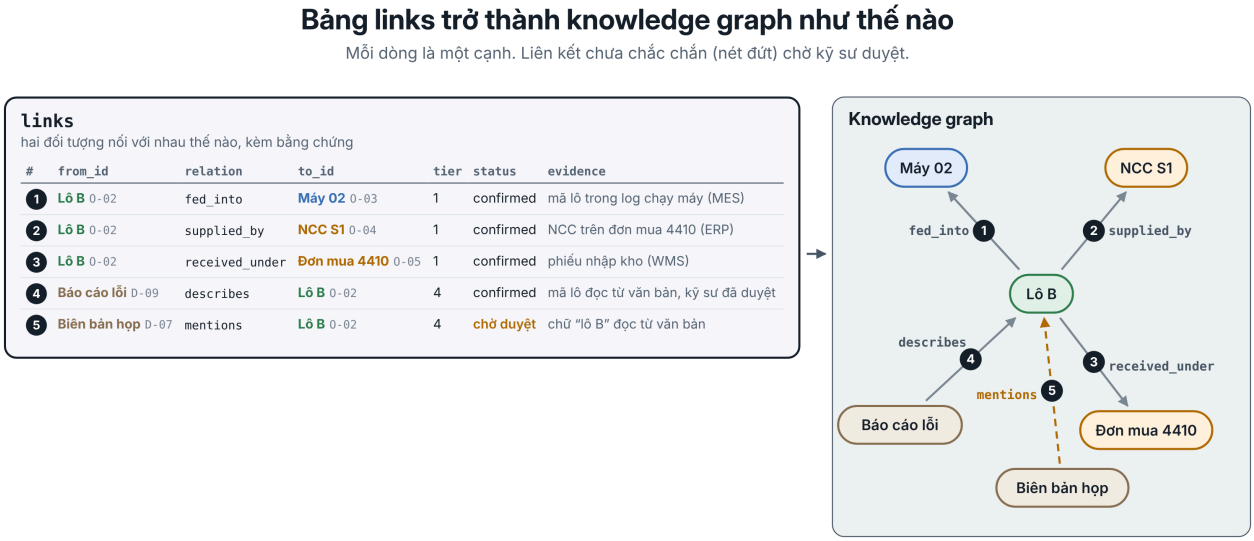


Hình 3. Sáu bảng lõi kèm dòng ví dụ. Mũi tên cho biết trường nào trỏ tới bảng nào. Danh sách trường đầy đủ ở bảng bên dưới.

BẢNG	TRƯỜNG CHÍNH	VÍ DỤ
`objects`	object_id, type, name, attributes (JSON)	Machine 02 (loại máy); Supplier S1 (loại nhà cung cấp); PO 4410 (loại đơn mua hàng); Customer C3 (loại khách hàng)
`aliases`	object_id, system, local_name, approved_by	Machine 02 ← "PLC2" trong PLC, "MC-02" trong MES; Supplier S1 ← "NCC-S1" trong ERP
`events`	event_id, type, object_ids, event_time, time_kind, source, source_record_id, payload (JSON)	Chu kỳ ép trên Machine 02, 13:58:04 (sensor); nhập kho lô B theo PO 4410 (giao dịch)
`documents`	doc_id, file_uri, type, source, chunk_id, page, text, extracted_codes, embedding	File thiết kế MH-20, trang 7, mã ["6203"]; biên bản họp 29/10 nhắc tới lô B
`links`	from_id, to_id, relation, tier, status, score, rule_version, evidence	lô B → Machine 02, fed_into, bậc 1; lô B → Supplier S1, supplied_by, bậc 1
`time_mappings`	source, offset, uncertainty, valid_from	PLC2, +38 s, ±2 s, từ 31/10

`time_kind` cho biết mốc thời gian mang nghĩa gì: *sensor* (đồng hồ của máy, được bù ở bước 3), *giao dịch* (lúc một bản ghi nghiệp vụ được ghi nhận) hoặc *hiệu lực* (lúc một bản vẽ, đơn giá hay quy định bắt đầu áp dụng).

Hình 4 cho thấy bảng `links` trở thành knowledge graph ở mục 2.9 như thế nào: mỗi dòng là một cạnh, và các liên kết còn chờ duyệt được tách riêng khỏi các liên kết đã xác nhận.



Hình 4. Mỗi dòng có số thứ tự trong `links` là cạnh mang cùng số trong graph. Liên kết nét đứt (bậc 4, chờ duyệt) chỉ được hiển thị như một gợi ý, không phải sự thật, cho tới khi kỹ sư duyệt.

Mô hình mở rộng mà không phải thay đổi

- **Loại đối tượng và quan hệ là cấu hình.** Chúng nằm trong danh mục loại và quan hệ (mục 3), không nằm trong schema. Thêm phòng Mua hàng nghĩa là thêm loại `supplier`, `purchase_order` và một file mapping.
- **Trường riêng của từng phòng ban nằm trong JSON.** Mã số thuế của nhà cung cấp, model của máy, hạn dùng của lô đều nằm trong `attributes` hoặc `payload`. Mỗi loại có thể kèm một bộ kiểm tra cho các trường quan trọng. Hàng trăm trường ở nhiều phòng ban, vẫn chỉ sáu bảng.
- **Một trường chỉ trở thành loại đối tượng dùng chung khi các phòng ban cần nối qua nó.** Nhà cung cấp là một loại vì Mua hàng, Chất lượng và Kho đều nhắc tới. Màu sơn của máy thì vẫn chỉ là thuộc tính.
- **Bảng theo từng phòng ban được dựng ở tầng sau.** Silver và Gold tạo các bảng như góc nhìn Mua hàng hay góc nhìn Chất lượng, nên lõi vẫn tổng quát mà báo cáo vẫn dễ truy vấn.

PHÒNG BAN	LOẠI ĐỐI TƯỢNG DÙNG CHUNG (BỘ KHỞI ĐẦU)
Sản xuất	máy, dây chuyền, trạm, lô, linh kiện, serial
Thiết kế	sản phẩm, mã linh kiện, phiên bản bản vẽ, BOM
Mua hàng, kho	nhà cung cấp, đơn mua hàng, phiếu nhập kho, vị trí kho
Chất lượng	lần kiểm tra, khiếu nại, báo cáo lỗi
Bảo trì	phiếu bảo trì, phụ tùng
Nhân sự	nhân viên, ca làm việc, chứng chỉ
Kinh doanh, logistics	khách hàng, đơn bán hàng, lô xuất hàng
Chung	cuộc họp, quyết định

Quan hệ chia làm ba nhóm. **Dòng sản phẩm:** `part_of`, `next_station`, `fed_into`, `processed_on`, `packed_in`, `shipped_in`. **Nghiệp vụ:** `supplied_by`, `received_under`, `ordered_in`, `delivered_to`, `performed_on`, `operated_by`, `specified_by`, `revision_of`. **Tri thức:** `describes`, `mentions`, `decided_in`. Tên quan hệ mới đi qua cùng quy trình AI đề xuất, kỹ sư duyệt như mọi registry khác.

5 AI agent và các tool

Agent không bao giờ truy vấn database trực tiếp. Nó lập kế hoạch các bước, gọi tool từ lớp truy vấn, đọc kết quả trả về và viết câu trả lời có nguồn cho từng con số. Tất cả tool đều chỉ đọc và đi qua cùng cơ chế phân quyền, audit log như người dùng thật.

TOOL	TRẢ VỀ
`trace_serial(serial)`	Toàn bộ hành trình của serial: lô đã dùng, trạm, thời gian, thùng, lô xuất hàng
`get_events(object, window)`	Event trên một đối tượng trong khoảng thời gian, kèm bản ghi gốc
`get_links(object, status)`	Đối tượng và tài liệu được liên kết, kèm bậc, trạng thái và điểm
`get_docs(object, at_time)`	Tài liệu tĩnh còn hiệu lực với đối tượng tại thời điểm đó, kèm nội dung chunk
`metrics(kpi, object, window)`	KPI từ Gold như cycle time, OEE, Cpk
`run_python(code)`	Phân tích trên dữ liệu đã lấy về (xu hướng, so sánh theo lô) trong sandbox không có mạng

Agent chỉ được dùng liên kết confirmed và inferred làm bằng chứng, và phải nói rõ đã dùng loại nào. Liên kết chỉ gợi ý chỉ được nhắc tới như hướng cần kiểm tra thêm.

6 Kịch bản: truy vết claim sau ba tháng

Claim về vào tháng 2; linh kiện được sản xuất tháng 11. Vì mọi bản ghi đã được liên kết ngay khi phát sinh, bức tranh tháng 11 vẫn còn đầy đủ.

#	DIỄN BIẾN	VỊ TRÍ TRONG KIẾN TRÚC
1	Claim CLM-0012 được ghi nhận: Part 0001 bị trả về vì tiếng ồn.	QMS → CDC → bước 1–4; claim nối với serial 0001 bằng khoá chính xác (bậc 1).
2	Người dùng mở claim trong Reports. `trace_serial(0001)` cho thấy Bearing 6203 lô B, được ép trên Machine 02 ngày 03/11 lúc 14:05.	Lớp truy vấn → knowledge graph. Các liên kết đã được tạo từ tháng 11 bởi bậc 1 và 2.
3	Bấm vào Machine 02 để xem event của giờ đó: cycle time tăng từ 31 s lên 42 s ngay sau lần nạp lô B lúc 13:52; lực ép đỉnh tăng 6 %.	Lớp truy vấn → lakehouse Silver (lịch sử). Khớp được mốc 13:52 là nhờ bước 3 đã bù độ lệch +38 s của PLC.
4	Bấm vào Bearing 6203 để xem tài liệu tĩnh còn hiệu lực tại ngày đó: bản vẽ, spec, kết quả IQC với Cpk 0.92 cho lô B.	`get_docs`; chunk bản vẽ được gắn bởi bậc 4 sau khi duyệt.
5	Ca tương tự: CLM-0009 (cùng họ bearing, đã xử lý bằng tăng lấy mẫu IQC), và một báo cáo lỗi cũ về tiếng ồn ổ bi hiển thị như hướng cần xem thêm.	CLM-0009 nối bằng bậc 1; báo cáo lỗi cũ là gợi ý bậc 5, đánh dấu chưa xác nhận.
6	Agent soạn nháp: "Nguyên nhân khả dĩ: bearing lô B (Cpk 0.92). Đã loại trừ bảo trì, người vận hành và nhiệt độ khu vực. CLM-0009 đã xử lý bằng tăng lấy mẫu IQC." Mỗi câu đều dẫn bản ghi gốc.	AI agent dùng `metrics`, `run_python` (so sánh theo lô) cùng event từ CMMS, HR và cảm biến.
7	Khoanh vùng: lô B đã đi vào 1.000 part, 42 thùng và 3 lô xuất hàng; 2 lô đã tới tay khách hàng.	Truy vết xuôi trên knowledge graph: lô → `processed_on` → `packed_in` → `shipped_in`.

Nếu không có thread, các bước 2 tới 7 nghĩa là gửi email cho sản xuất, chất lượng, mua hàng và logistics, mỗi bên trả lời bằng cách đặt tên và đồng hồ của riêng mình.

7 Hợp nhất hai bản phác thảo của đội

Ý TƯỞNG	TỪ	NẰM Ở ĐÂU TRONG KIẾN TRÚC
Connector kéo dữ liệu, chỉ đọc, mỗi loại nguồn một connector	Sơ đồ pipeline	Ingest, cửa hệ thống
Endpoint POST chung + RabbitMQ	Báo cáo ý tưởng	Ingest, cửa tài liệu
Bản sao gốc và chạy lại	Sơ đồ pipeline	Bronze
Chuẩn hoá thời gian, tên thiết bị, ID	Cả hai	Bước 1–3, mỗi bước một registry
Thang liên kết năm bậc, confirmed / inferred / gợi ý	Báo cáo ý tưởng	Bước 4
Chunk tài liệu, LLM phân loại, kỹ sư duyệt	Báo cáo ý tưởng	Nhánh tài liệu ở bước 1 và bậc 4
Tài liệu tĩnh trên node, log trên cạnh	Báo cáo ý tưởng	Bảng lõi (tài liệu gắn vào đối tượng, event gắn vào đối tượng + thời điểm); vẽ trên cạnh ở Plant view

Ý TƯỞNG	TỪ	NẰM Ở ĐÂU TRONG KIẾN TRÚC
Redis, MinIO	Báo cáo ý tưởng	Redis cho trạng thái live; MinIO cho Bronze và file
Trạng thái live, lakehouse, knowledge graph	Sơ đồ pipeline	Ba lớp dữ liệu tối ưu cho việc đọc
Lớp truy vấn có phân quyền; luồng federated	Sơ đồ pipeline	Lớp truy vấn
AI đề xuất, kỹ sư duyệt, có version	Cả hai	Control plane
AI agent qua MCP tool	Cả hai	Ứng dụng

8 Phần demo và kiến trúc mục tiêu

Demo cho thấy hệ thống làm được gì trên dữ liệu thực tế. Kiến trúc cho thấy hệ thống mở rộng ra sao. Demo chạy cùng logic với kiến trúc mục tiêu, chỉ trên một stack nhỏ hơn.

PHẦN	TRONG DEMO	Ở QUY MÔ NHÀ MÁY
Nguồn	File mẫu cố ý "lộn xộn": tên máy khác nhau, PLC lệch giờ, một file thiết kế PDF, một báo cáo lỗi không ghi mã linh kiện	Hệ thống đang chạy thật
Ingest, Bronze	Phát lại file theo thứ tự thời gian	Connector, Kafka, RabbitMQ, MinIO
Bước 1–4	Một script Python, bậc 1–4	Job Flink, thêm bậc 5
Registry	File YAML và một bảng bí danh	Như demo, thêm màn duyệt và lịch sử Git
Bảng lõi	PostgreSQL	PostgreSQL + TimescaleDB + pgvector
Các lớp đọc	Truy vấn thẳng trên bảng lõi	Redis, lakehouse, Neo4j
Lớp truy vấn, AI	FastAPI + MCP tool + agent	Như demo, thêm Keycloak và audit log
Web app	Plant, Product, Assistant; Reports dựng sẵn	Đủ năm màn

9 Tổng hợp công nghệ

LỚP	CÔNG NGHỆ	LÝ DO
Edge	OPC UA, Mosquitto (MQTT)	Chuẩn phổ biến trong nhà máy; lưu tạm khi mất mạng
Bắt thay đổi DB	Debezium	Đọc thay đổi của MES/WMS từ log, không tạo tải lên nguồn
Lấy theo lịch	Airflow	Lên lịch gọi API, lấy CSV, file xuất hằng đêm, chạy lại
Event bus	Apache Kafka	Có thứ tự, bền vững, nhiều bên đọc cùng lúc
Nhận tài liệu	FastAPI, RabbitMQ	Cửa tải lên đơn giản và hàng đợi job theo từng file
Lưu bản gốc	MinIO, Apache Iceberg	Object storage tại chỗ; bảng có thể chạy lại và quản lý version
Xử lý	Apache Flink	Join luồng có trạng thái cho định danh, thời gian và liên kết
Parse tài liệu	Docling	Parse PDF và bản scan theo bố cục, chia thành chunk
LLM	Claude	Trích mã, đề xuất mapping, chạy agent; có thể thay bằng model đặt tại chỗ
Kho lõi	PostgreSQL, TimescaleDB, pgvector	Quan hệ, chuỗi thời gian và embedding trong một database
Trạng thái live	Redis	Đọc nhanh tới mức giây cho màn hình nhà máy
Lakehouse	Iceberg, dbt, Trino	Lịch sử, model KPI có kiểm thử, SQL trên bảng lớn
Graph	Neo4j	Truy vết nhiều bước và khoanh vùng nhanh
Lớp truy vấn	FastAPI, MCP server, Keycloak	Một API cho cả app và AI; vai trò và đăng nhập

LỚP	CÔNG NGHỆ	LÝ DO
Giám sát, cảnh báo	Prometheus, Grafana	Sức khoẻ pipeline; dashboard và luật cảnh báo
Cấu hình	Git (YAML), PostgreSQL	Mọi thay đổi mapping đều được duyệt và có thể hoàn tác
Web app	React, React Flow	Màn hình graph tương tác

10 Các quyết định còn mở

- **LLM chạy ở đâu.** Nếu dữ liệu nhà máy không được ra ngoài, agent và phần trích mã chuyển sang một model mã nguồn mở đặt tại chỗ. Kiến trúc không đổi; chất lượng câu trả lời cần được kiểm thử.
- **Đo chất lượng liên kết thế nào.** Mỗi bậc cần được kiểm tra độ chính xác trên mẫu đã gán nhãn trước khi tin dùng, và độ rộng cửa sổ thời gian của bậc 3 cần chỉnh theo từng trạm.
- **Quy tắc truy cập.** Vai trò nào được xem dữ liệu HR, chi phí, nhà cung cấp; và agent kế thừa quyền của người hỏi (khuyến nghị) hay có quyền riêng.
- **Quy mô xử lý.** Flink phù hợp khi có nhiều line và cảm biến tần số cao. Một pilot ở một nhà máy có thể chạy bước 1–4 bằng service Python đơn giản trước.